

AI Worklog

Tools, models, planning process, responsibilities, corrections, verification, and remaining risk for the ops-mcp project.

Project: **ops-mcp**

Type: **AI-native commerce-ops MCP server**

Date: **August 2026**

1. AI Tools & Models Used

CHAT / PLANNING

Claude (Sonnet 5, free plan)

IMPLEMENTATION

**Claude Code on OpenRouter
(poolside/laguna-s-2.1:free)**

Claude (chat) handled all planning, MCP tool/architecture design, client-communication drafting, and review of implementation output against the written spec.

Claude Code — configured to run on `poolside/laguna-s-2.1:free` via OpenRouter — handled all codebase implementation: schema, data access layer, tool handlers, server, tests, migration, and deployment configuration. This is a non-default configuration: Claude Code's tooling harness driven by a non-Anthropic model. No operational differences were observed versus a standard session; file edits, typecheck, and test runs behaved identically.

2. Why These Models Were Selected

Chat was used for anything that benefited from sustained conversational reasoning without needing to touch the filesystem: scoping the single workflow to build, designing the MCP tool surface (names, schemas, description strings written for an AI consumer), working through client feedback, and making explicit tradeoff calls (e.g., the idempotency-key design). Keeping this separate from implementation made it possible to review Claude Code's output against an explicit external spec rather than trusting in-context memory.

Claude Code was used for implementation because it's built for sustained, tool-using work against a real filesystem and test suite. The choice of the underlying model (Laguna S 2.1 rather than a Claude model) was a deliberate cost/experimentation choice on the developer's part; it was monitored for quality via the same review process that would apply to any model — every implementation claim was checked against actual command output, not accepted on the model's own report.

3. Planning & Work Breakdown

The project was scoped entirely in chat before any code was written:

- Choosing one well-bounded workflow — **"charged but order failed"**, caused by an expired inventory hold — over broader feature coverage.
- Designing the read/write-separated MCP tool surface.
- Specifying exact seed data, including deliberate edge cases: an already-refunded order, a cancelled order, and a "decoy" order that looks broken but isn't.

A `handoff.md` document was established as the single working spec between chat and Claude Code. Every architecture decision made in chat is written there as a concrete, actionable item before Claude Code touches code, and every completed piece of work is logged back with a changelog entry. Chat owns architecture and decisions; Claude Code owns all codebase changes; nothing gets implemented that wasn't first written down.

On the Claude Code side, work was further broken down using design-spec and implementation-plan documents (under `docs/superpowers/`) written before each implementation task — e.g., a specific design spec for the

transactional-coupling fix laid out the problem statement, the before/after data flow, and a testing plan before any code changed. A `memory.md` file was maintained as a running session anchor (goal, constraints, live credentials, architecture, decisions, file inventory, verification status), updated in place so context persisted across a long implementation session.

4. Division of Responsibilities

Party	Responsibilities
Chat Claude Sonnet 5	Scope and architecture decisions, MCP tool design, all client-facing email drafts, reviewing Claude Code's output against the written spec, catching gaps between what was requested and what was delivered, maintaining <code>handoff.md</code> .
Claude Code Laguna S 2.1	Implementation of everything in <code>handoff.md</code> 's Pending Changes — schema, data access layer, tool handlers, server, Postgres migration, deployment config, and test suite; wrote and maintained its own design specs and session-memory file.
Developer	Final review and testing judgment; ran verification commands directly against the live deployment rather than trusting either AI's self-report; chose and executed the actual Render deployment; made the final call on every design tradeoff surfaced in chat; decided when client feedback required a scope change versus a clarification.

5. Prompts, Instructions & Context Supplied

The most consequential input wasn't a single prompt — it was the client's own reply to an initial scope-and-questions email, which specified atomicity, idempotency, and oversell-safety requirements in precise technical language. That reply was treated as a primary spec: each requirement was translated in chat into an unambiguous engineering task, with tradeoffs made explicit, before being handed to Claude Code.

For idempotency specifically, three implementation options were worked out and compared in chat before one was locked and written into `handoff.md` as the target design:

OPTION A

Agent-generated key

OPTION B

Server-derived key

OPTION C

Reject-on-retry

This was treated as a real design decision requiring judgment, not left to the implementation step to decide.

6. AI Suggestions Corrected, Rejected, or Substantially Changed

1. Non-transactional audit logging — the main correction

After the first implementation round, Claude Code's own description of the audit logging it had built was "best-effort (DB errors caught, never blocks operation)." Checking that against the client's actual stated requirement —

the audit record for an approved mutation must share the same transaction as the mutation — showed the implementation didn't meet it: best-effort logging is the direct opposite of transactional coupling. A failed `action_log` insert would leave a committed mutation with no audit trail — a real data-integrity failure in an audit context. This was caught in chat by reading Claude Code's own summary carefully rather than accepting a "done" status at face value, sent back as a precise written correction, and re-verified against live Supabase. Both the mutation-success path and the idempotency-key storage were folded into the same fix, since they had the identical class of bug.

2. A test written against the wrong seed order

During the transactional-fix implementation, the first version of the new verification test used order `A1005` to check a successful refund path — but `A1005`'s payment status is "authorized," not "captured," so the refund handler correctly rejects it before ever reaching the code path the test was meant to exercise. This was caught immediately by the test suite itself (26/30 passing, 4 failures traced to `A1005`) rather than by manual review. Corrected by switching to `A1001` (a delivered order with a captured payment), after which all 30 tests passed.

3. A minor security practice deviation, self-corrected

During the first `git push`, Claude Code embedded a GitHub personal access token directly in the origin remote URL for one push, then reset the remote to a token-free URL immediately after and used the token inline for subsequent pushes instead. The token was not committed or persisted in git config history, but embedding a credential in a remote URL — even temporarily — isn't best practice. Noted here because it's a small example of exactly the kind of thing worth watching for when supervising AI-driven git operations.

4. A demo-reset tool silently defeating the audit trail

Reviewing the seed script's `--reset` flag (added to let the developer restore clean demo data before recording) surfaced that it truncated the `action_log` table along with everything else. Functionally this worked fine — no error, no test failure — but it violated the client's requirement in spirit: they'd asked to "preserve workflow state **and** audit history" as two distinct things. This wasn't caught by any test (no test asserted that audit rows survive a reset); it was caught by re-reading the client's original wording. Fixed by excluding `action_log` specifically from the reset's TRUNCATE statement, while correctly leaving `idempotency_keys` in scope for reset — those rows are only meaningful relative to the business-state mutation they were generated against, so they become orphans regardless.

7. How AI-Generated Work Was Verified

- Every MCP tool was exercised with real `curl` calls against the actually-running server — locally first, then again against the public Render URL — to confirm the MCP protocol layer worked end-to-end (`initialize` handshake, `tools/call`), not just that unit tests passed in isolation.
- Both resolution paths (reconfirm, refund) and all edge/rejection cases (already-refunded order, cancelled-pre-capture order, decoy order, unknown order ID) were individually exercised and their actual output read.
- When `handoff.md` updates were internally inconsistent (one section describing a live Postgres deployment while another still said "not yet deployed, 14/14 tests"), the inconsistency was caught by reading the full document and reconciled explicitly.
- The audit-transaction gap was caught by close reading of Claude Code's own implementation description — a reminder that a passing test suite doesn't guarantee a specific stated requirement was met if the suite wasn't written to check for that exact property.

- On the implementation side: local tests run via `pg-mem` — 30/30 passing after the transactional fix, up from a caught regression of 26/30 — plus `tsc --noEmit` and a full `npm run build` confirmed clean. A temporary verification script ran directly against the live Supabase instance (11/11 manual checks: transactional commit of audit+idempotency, idempotent replay, oversell rejection, reconfirm happy path, shipment verification), then was deleted after use, with demo data restored via the standard (non-destructive) seed run afterward.

8. Remaining Risks or Unfinished Work

SUMMARY

No blocking issues for the single-agent demo use case. Three documented production gaps (concurrency coverage, a read-inside-transaction inconsistency, and rejection-path audit writes) are accepted tradeoffs; the cold-start is operational and the idempotency-key TTL is a known production enhancement.

Render free-tier cold start

The deployed instance sleeps after ~15 minutes idle; the first request after a gap can take 30-50 seconds. Documented, not yet mitigated with a keep-warm ping — still an open decision before final demo recording.

Production seed reset completed

The live database previously had real mutation history from testing (e.g., order `A1023` already reconfirmed). The `--reset` flag — confirmed to correctly preserve `action_log` while resetting business state — has now been run against production, restoring clean demo state ahead of final recording.

No concurrency test for the oversell fix

The atomic conditional stock update was verified functionally (correctly rejects when stock is insufficient) but not exercised under genuinely concurrent simultaneous requests — the atomicity guarantee is a property of the single SQL statement, not something a race-inducing test currently confirms.

Pre-existing, unfixed read-inside-transaction inconsistency

Inside `reconfirmOrder()`'s transaction, a shipment-existence check reads via the connection pool rather than the transaction's own client connection — meaning that specific read could see stale data under real concurrency. Identified but not fixed, since it doesn't affect the single-agent demo use case; worth a note for anyone extending this toward genuine concurrent production use.

No TTL on idempotency keys

They persist indefinitely; a production system would expire them (Stripe uses 24h). Not implemented — acceptable for a small, fixed demo dataset, called out explicitly as a known production gap.

Rejection-path audit/idempotency writes remain best-effort

By necessity — once a mutation's transaction has rolled back, there's no longer a transaction to couple a rejection record to, so those specific writes are standalone and could theoretically be lost on a transient DB error. This is a considered tradeoff, not an oversight.